

DEEP LEARNING

Studying Notes (ML Analogy):



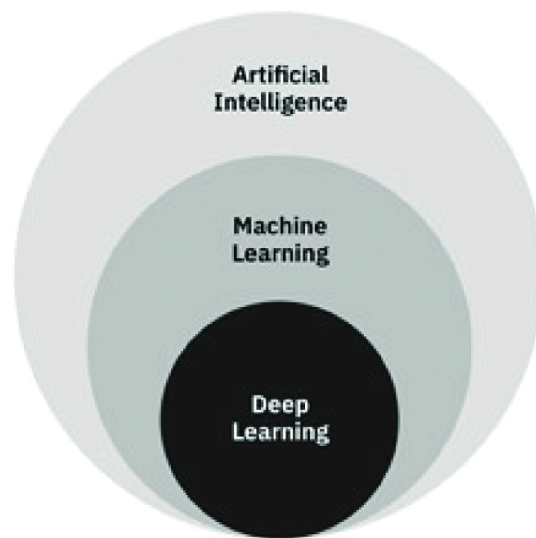
Process: When a student studies notes, they read through the material, memorize key concepts, and practice problems. Over time, they learn to recognize patterns in the information, apply rules, and make decisions based on what they've learned.

Learning Approach: This is akin to **Machine Learning**. The student (ML model) is trained on specific examples (training data) and, through repetition and practice (learning algorithms), they generalize the knowledge to apply it to new problems (test data). The student doesn't need to understand the material at a deep level but can still perform well by recognizing patterns and applying learned rules.

Baby Recognizing Faces (DL Analogy):



- **Process:** A baby learns to recognize faces by seeing them repeatedly, starting from basic shapes and features. Over time, the baby learns to distinguish subtle differences between faces and recognizes them even in different contexts (e.g., different lighting, angles).
- **Learning Approach:** This is analogous to **Deep Learning**. The baby (DL model) starts with basic observations (simple layers in a neural network) and builds increasingly complex representations of faces (deeper layers in the network). Unlike the student, the baby doesn't just memorize features but develops a deep understanding of what makes each face unique, allowing for more sophisticated recognition even in varied situations.



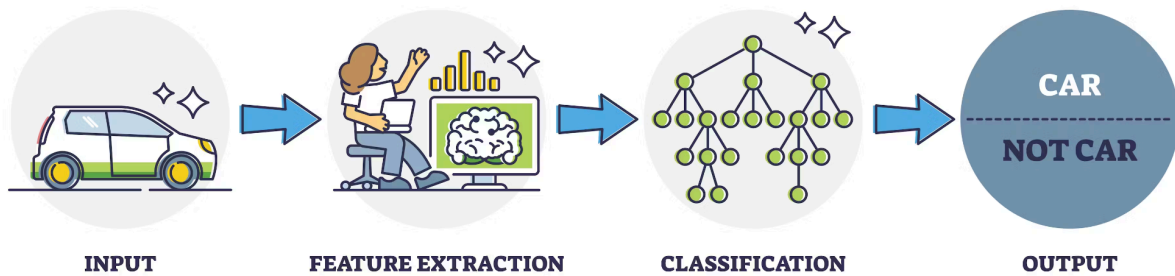
Machine Learning

- Good results with small data sets
- Quick to train a model
- Need to try different features and classifiers to achieve best results
- Accuracy plateaus

Deep Learning

- Requires very large data sets
- Computationally intensive
- Learns features and classifiers automatically
- Accuracy is unlimited

MACHINE LEARNING



DEEP LEARNING



Introduction to Deep Learning:

- Deep learning is a branch of machine learning and AI which is based on **neural networks (NN)**. It is capable of learning complex patterns and relationships within data. In deep learning, **we don't need to explicitly program everything.**

Deep Learning Applications:

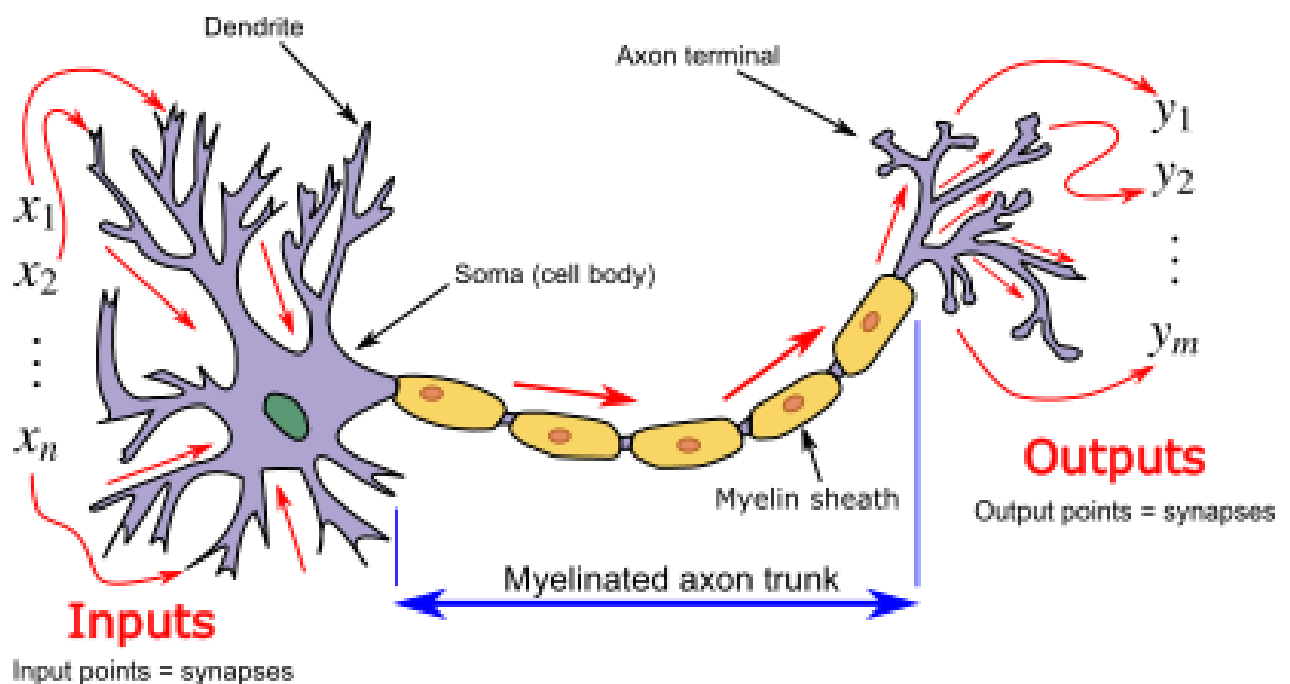
1. Image Recognition in Healthcare:

- DL models, such as Convolutional Neural Networks (CNNs), are used to analyze medical images like X-rays or MRIs, enabling the early detection of diseases such as cancer.

2. Speech Recognition in Virtual Assistants:

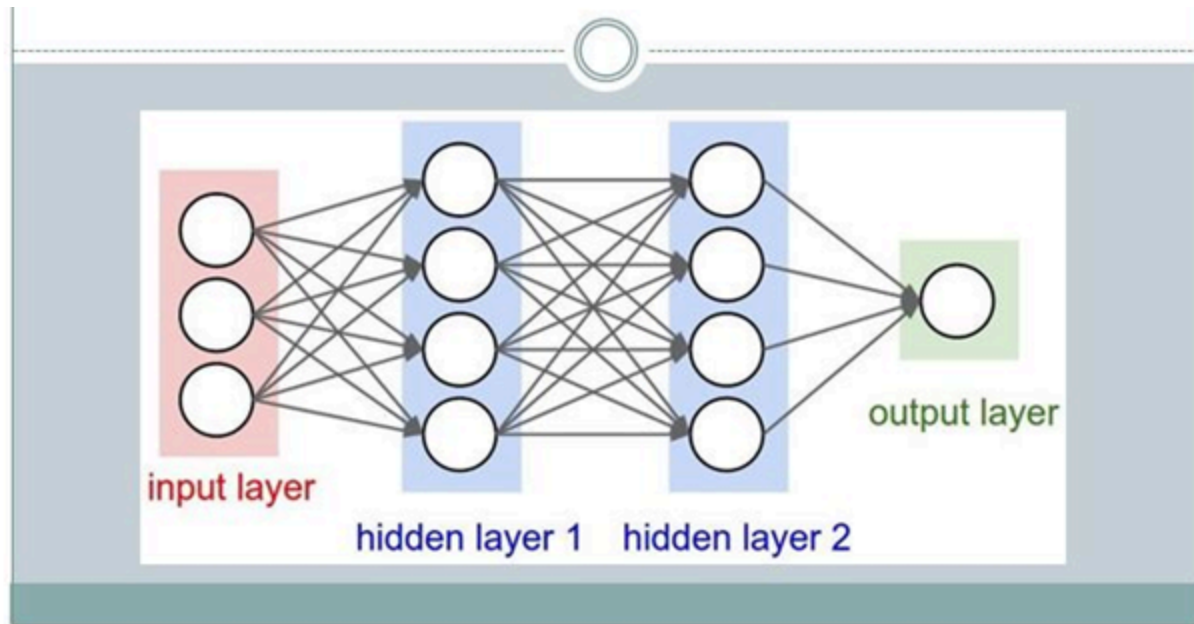
- DL models power speech recognition systems in virtual assistants (e.g., Siri, Alexa), allowing them to understand and respond to voice commands with high accuracy.

Neural Networks:

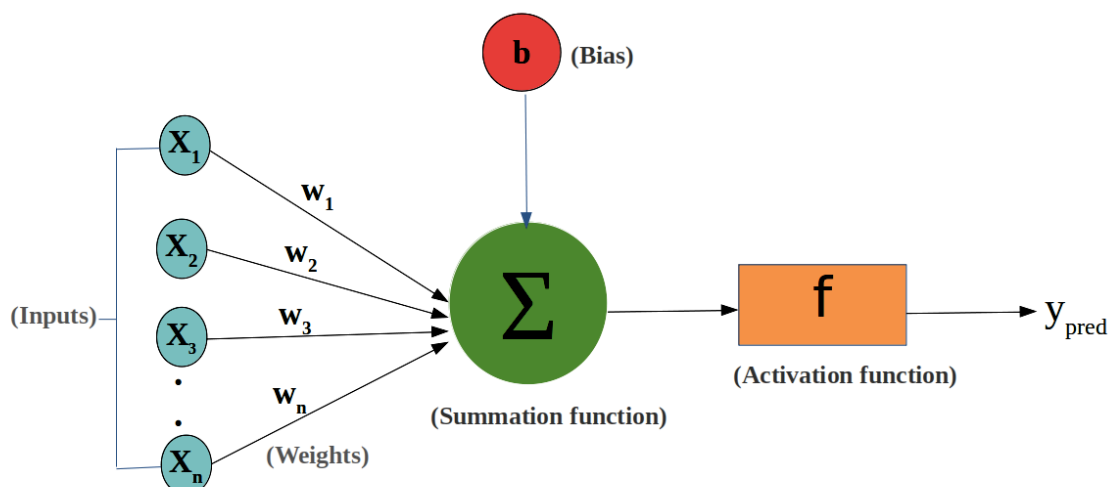


Neural Networks are a class of machine learning models inspired by the human brain's structure and function. They are composed of interconnected units called neurons, organized into layers. The key layers in a neural network are:

1. **Input Layer:** Receives the initial data and passes it onto the next layer.
2. **Hidden Layers:** Intermediate layers where computations are performed. These layers extract features and patterns from the input data.
3. **Output Layer:** Produces the final prediction or classification.



Now, let's say you're building a neural network to predict house prices based on features like square feet, age, number of bedrooms, and location. Here's how weights come into play:



1. Input Layer:

- Each input node represents a feature of the house, such as square footage, number of bedrooms, etc.
- For example, let's say you have three input nodes representing square foot, number of bedrooms, and distance to the city center.

2. Hidden Layer:

- Each node in the hidden layer takes input from all the nodes in the input layer.
- Each input connection has an associated weight, which determines the strength of the connection between the input node and the hidden node.
- These weights represent the importance of each feature in predicting the house price.
- For instance, if the weight associated with the square footage input node is high, it means that square footage has a significant impact on the predicted house price.

3. Output Layer:

- The output layer produces the final prediction, which in this case, is the predicted house price.
- Each node in the output layer receives input from all the nodes in the hidden layer, weighted by the connections' weights.
- The output node aggregates these weighted inputs to produce the predicted house price.
-

TYPES OF NEURAL NETWORKS

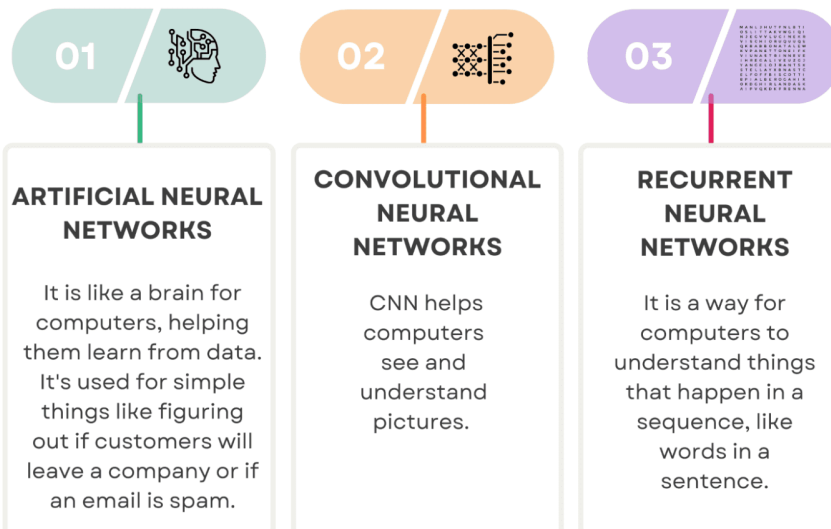
1. Artificial Neural Networks (ANN) for Text,number Data:
 - ANNs can be used for text data processing tasks such as sentiment analysis, document classification, and text generation.
 - For tasks where the sequential nature of text is important, recurrent neural networks (RNNs) or their variants like Long Short-Term Memory (LSTM) networks are often preferred.
2. Convolutional Neural Networks (CNN) for Image Data:
 - CNNs are ideally suited for image data due to their ability to automatically learn hierarchical features from raw pixel data.
 - They excel at tasks such as image classification, object detection, image segmentation, and image generation.
3. Recurrent Neural Networks (RNN) for sequential Data: eg: sound, video..
 - RNNs, especially variants like LSTM and Gated Recurrent Unit (GRU), are commonly used for sequential data processing, including sound data.
 - Sound data often consists of time-series information, making RNNs suitable for tasks like speech recognition, sound classification, and music generation.
4. An LSTM (Long Short-Term Memory)

network is a type of RNN recurrent neural network that is capable of handling and processing sequential data.

What is LSTM and why it is used?

LSTMs are predominantly used to learn, process, and classify sequential data because these networks can learn long-term dependencies between time steps of data. Common LSTM applications include sentiment analysis, language modeling, speech recognition, and video analysis.

NEURAL NETWORK TYPES

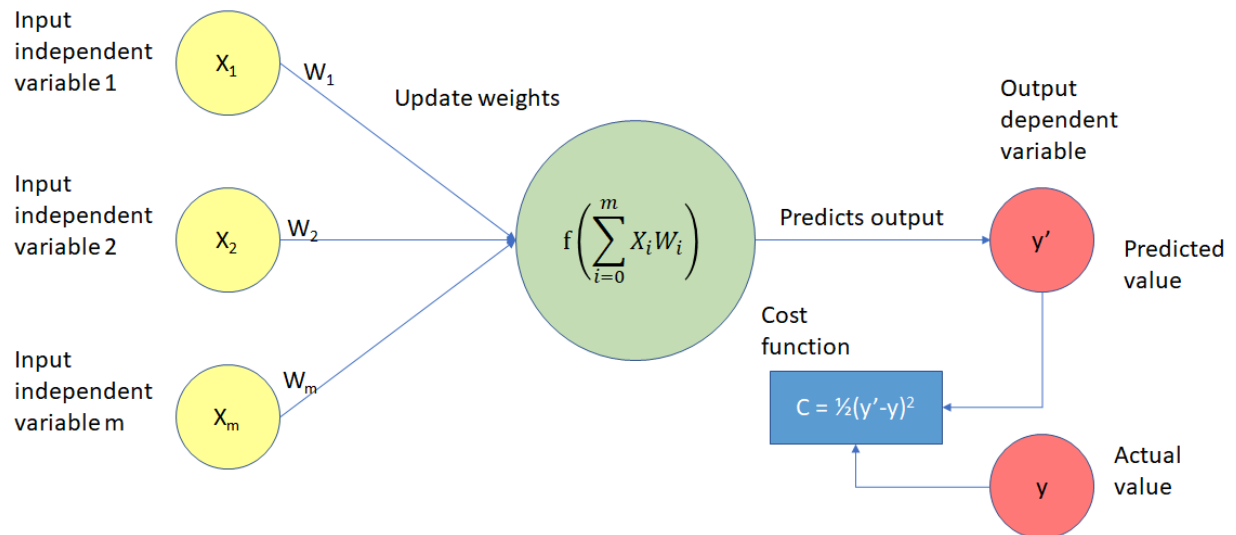


intelligence

Artificial intelligence- brain, artificial neural networks

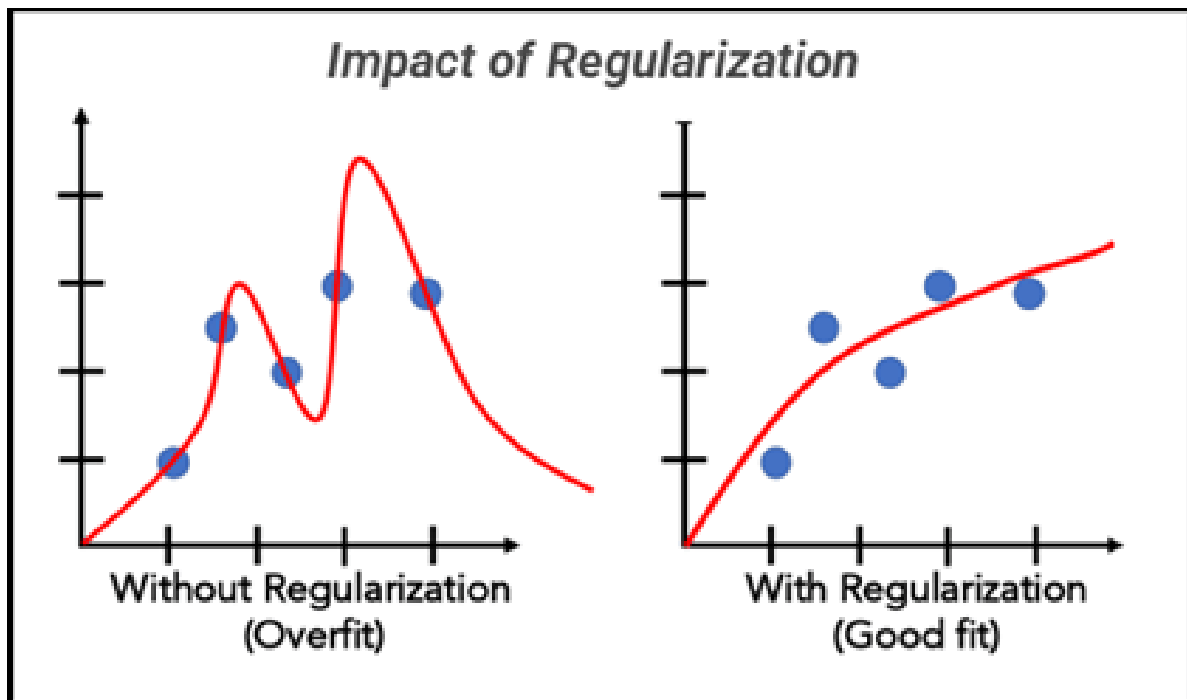
Deep learning -

Let's break down the process of weight calculation in a neural network in simpler terms:



1. Initialization: When you create a neural network, the weights connecting each input to each neuron in the hidden layer(s) are assigned random values. These initial weights are like starting points for the network to learn from.
2. Forward Pass: During the forward pass, input data is fed into the network. Each neuron in the hidden layer(s) takes these inputs and combines them with the weights connected to it. This combination is done by multiplying each input by its corresponding weight and summing up these products. Think of it as each input having its own importance (the weight), and the neuron deciding how to use these inputs to produce an output.
3. Activation Function: After this weighted sum is calculated, it's passed through an activation function. This function introduces non-linearity into the output of the neuron. Common activation functions include ReLU, sigmoid, and tanh. This activated output is then passed to the next layer.
4. COST /LOSS Calculation: Once the data has propagated through the network and the final output is produced, it's compared to the actual desired output. The difference between the predicted output and the actual output is calculated using a loss function.
5. Backpropagation: Backpropagation is a method used to update the weights of the neural network to reduce the loss.
6. Gradient Descent: Gradient descent is the optimization algorithm used to adjust the weights based on the gradients calculated during backpropagation. It essentially tells the network which direction to move the weights to reduce the error.
7. Weight Update: The weights are updated based on the gradients calculated using gradient descent. The amount by which each weight is adjusted is determined by a parameter called the learning rate. This process is repeated iteratively for each batch of training data until the network's performance improves.

REGULARIZATION

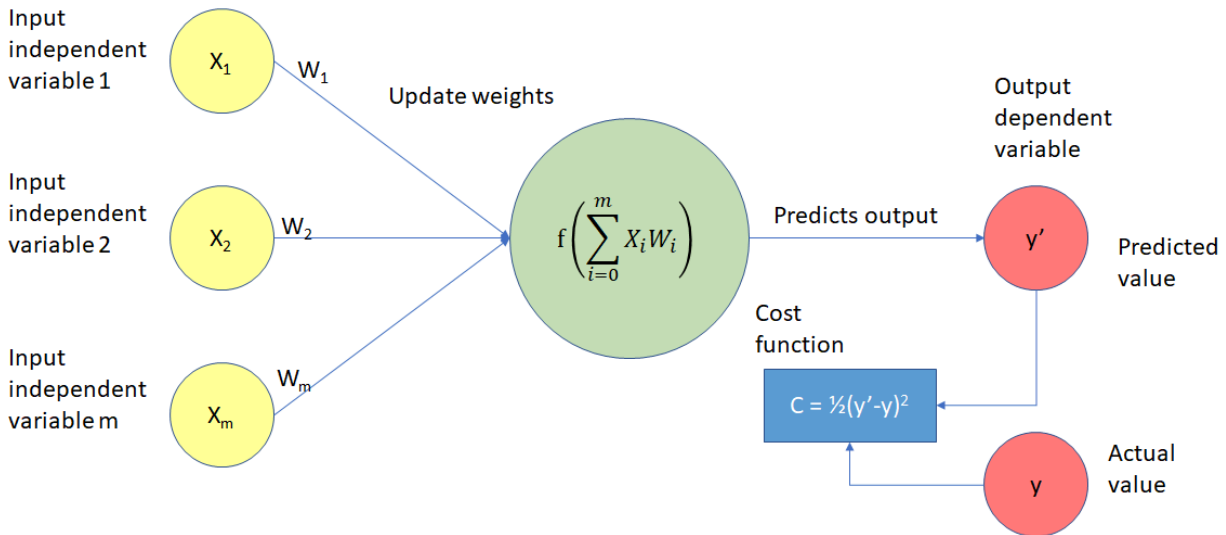


? OVERFIT

? UNDERFIT

Regularization is a set of methods for reducing overfitting in machine learning models.

Typically, regularization trades a marginal decrease in training accuracy for an increase in generalizability. (good fit)



L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$

1. L2 Regularization (Ridge Regression):

- Also known as weight decay, L2 regularization adds a penalty term to the loss function proportional to the squared magnitude of the weights.

- It discourages large weight values and encourages a smoother decision boundary.
- The regularization term is typically controlled by a hyperparameter λ (lambda).

2. L1 Regularization (Lasso Regression):

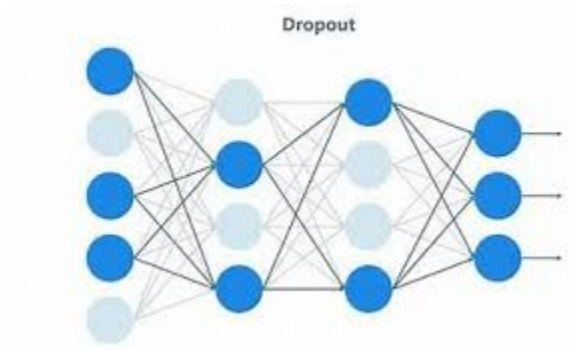
- L1 regularization adds a penalty term to the loss function proportional to the absolute value of the weights.
- It encourages sparsity in the model by driving some of the weights to exactly zero, effectively performing feature selection.
- L1 regularization is useful when dealing with high-dimensional data with many irrelevant or redundant features.

3. Elastic Net Regularization:

- Elastic Net regularization combines L1 and L2 regularization by adding both penalties to the loss function.
- It provides a balance between L1 and L2 regularization, allowing for feature selection while also stabilizing the model.

4. Dropout:

- Dropout is a technique specific to deep neural networks where randomly selected neurons are ignored during training with a certain probability.
- It prevents complex co-adaptations among neurons and encourages the network to learn more robust features.
- Dropout acts as an ensemble technique by training multiple subnetworks, reducing overfitting.



5. Early Stopping:

- Early stopping is a simple yet effective regularization technique where training is stopped when the performance on a validation set starts to degrade.
- It prevents overfitting by stopping training before the model starts to memorize the training data too closely.

6. Data Augmentation:

- Data augmentation involves generating new training examples by applying transformations like rotation, translation, scaling, and cropping to the existing data.
- By augmenting the training data with variations of the original samples, data augmentation helps increase the diversity of the training set and reduce overfitting.

7. Batch Normalization:

- Batch normalization is a technique used to normalize the activations of each layer in a neural network by adjusting and scaling the activations.
- It helps stabilize the training process, reduces internal covariate shift, and acts as a form of regularization.